COMP2022 Models of Computations: Propositional Logic

Contents

1 Syntax **3**

1.1 Recursive definition 4

2 Semantics **5**

2.1 Truth values and tables 5

2.2 Terminology 7

2.3 Conditional 7

2.4 Connection with natural language 8

3 Validity and satisfiability **9**

3.1 Valid formulas 9

3.2 Satisfiable formulas 9

3.3 Satisfiability problem 10

4 Logical Consequences **10**

5 Logical Equivalence **12**

6 Normal forms and modeling problems in logic **14**

6.1 Normal forms: NNF 14

6.2 Normal forms: CNF 14

7 The problem of finding cliques **16**

7.1 Solving CLIQUE with logic 17

7.2 Running Example 17

Logic is used for representing, computing and reasoning. We study the following two logic.

1. **Propositional logic.** About propositions, e.g., statements such as conditionals in programming, digital circuits.
2. **Predicate logic (aka First-order logic).** About relations and functions, e.g., the correctness of programs, knowledge-bases in AI, declarative programs, database queries.

A logic is made up of the following.

1. **Syntax** tells us the structure of expressions, or the rules for putting symbols together to form an expression.
2. **Semantics** refers to the meaning of expressions, or how they are evaluated.
3. **Deduction** is a syntactic mechanism for deriving new true expressions from existing true expressions.

We can write propositional logic formulas in most programming languages. In Python and Java these are called *Boolean expressions*. They are usually used as conditions for control flow, e.g., inside a conditional statement or a loop.

Definition (Proposition). A *proposition* is a sentence that declares a fact that is either true, or false, but not both.

1 Syntax

We will use another syntax which is the standard in computer science and mathematics.

Name	Prop. Logic	`Python`	`Java`
conjunction	$\wedge$	`and`	`&&`
disjunction	$\vee$	`or`	`||`
negation	$\neg$	`not`	`!`
implication	$\rightarrow$		
bi-implication	$\leftrightarrow$	`==`	`==`
top/verum	$\top$	`True`	`true`
bottom/falsum	$\bot$	`False`	`false`
atoms	$p, q, r, \ldots$	Boolean variables	Boolean variables
formulas	$F, G, H, \ldots$	Boolean expressions	Boolean expressions

We now precisely define the syntactic rules for defining formulas of Propositional Logic.

Definition. An *atom* is a variable of the form $p_1, p_2, p_3, \ldots, p, q, r, \ldots$ A *formula* is defined by the following recursive process.

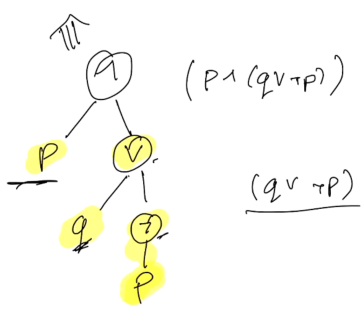
1. Every atom is a formula.
2. If F is a formula then $\neg F$ is a formula.
3. If F and G are formulas then so are $(F \vee G)$ and $(F \wedge G)$.

1.1 Recursive definition

This is another example of a recursive definition.

- The base case specifies the simplest objects, and the recursive cases specify how to build complex objects from ones we already built.
- Think of the definition as a construction manual.
- Let us build $(p \wedge (q \vee \neg p))$ using this definition.

We can also represent a formula as a tree.



The importance of this mathematical formula:

1. It specifies what we mean by a propositional formula. If we disagree on whether something is a formula, we just consult the definition.
2. It allows one to design algorithms that process/manipulate formulas using recursion as follows:
   - The base case is rule 1 of the definition.
   - The recursive case are rules 2 and 3 of the definition.
3. One can prove things about propositional formulas and about code that processes formulas.

Example. A formula which occurs in another formula is called a *subformula*. Give a recursive process that defines the set **SubForms**(G) of all subformulas of G.

Solution. The set of subformulas of a formula G is denoted **SubForms**(G), and is defined as follows:

1. If G is an atom, then **SubForms**$(G) = \{G\}$.
2. If G is a formula of the form $\neg F$, then **SubForms**$(G) = \{G\} \cup$ **SubForms**(F).
3. If G is a formula of the form $(E \vee F)$ or $(E \wedge F)$ then **SubForms**$(G) = \{G\} \cup$ **SubForms**$(E) \cup$ **SubForms**(F).

We use abbreviations:

- $(F_1 \vee F_2 \vee F_3)$ instead of $((F_1 \vee F_2) \vee F_3)$.
- $(F_1 \wedge F_2 \wedge F_3)$ instead of $((F_1 \wedge F_2) \wedge F_3)$.

E.g., we may write $(p \vee q \vee \neg r) \wedge r$ instead of $(((p \vee q) \vee \neg r) \wedge r)$.

2 Semantics

2.1 Truth values and tables

Definition (Semantics). *Semantics* refers to how we derive the value of a formula based on the values of its atomic subformulas.

- The elements of the set $\{0, 1\}$ are called *truth values*.
- We read 1 as "true", and 0 as "false".
- After we *assign* truth values to atoms we can give truth values to formulas.
- Because of the recursive structure of formulas, we only need to give the rules for each connective ($\wedge$, $\vee$, $\neg$).

The formula $(F \wedge G)$ is true if and only if both the formulas F and G are true.

F	G	$(F \wedge G)$
0	0	0
0	1	0
1	0	0
1	1	1

The formula $(F \vee G)$ is true if and only if either F or G or both are true.

F	G	$(F \vee G)$
0	0	0
0	1	1
1	0	1
1	1	1

Facts:

- An *assignment* is a function α from atoms to truth values, i.e., "row of a truth table".
- On three variables there are $2^3 = 8$ assignments.
- On four variables there are $2^4 = 16$ assignments.

Definition (Semantics).

An *assignment* is a function α from atoms to truth values.

The *truth value* $\mathrm{tv}(F, \alpha)$ of a formula F under assignment α is defined by the following recursive process:

1. $\mathrm{tv}(p, \alpha) = \alpha(p)$ for every atom p.
2. $\mathrm{tv}(\neg F, \alpha) = \begin{cases} 0 & \text{if } \mathrm{tv}(F, \alpha) = 1, \\ 1 & \text{if } \mathrm{tv}(F, \alpha) = 0. \end{cases}$
3. $\mathrm{tv}(F \wedge G, a) = \begin{cases} 1 & \text{if } \mathrm{tv}(F, \alpha) = 1 \text{ and } \mathrm{tv}(G, \alpha) = 1, \\ 0 & \text{otherwise.} \end{cases}$
4. $\mathrm{tv}(F \vee G, \alpha) = \begin{cases} 1 & \text{if } \mathrm{tv}(F, \alpha) = 1 \text{ or } \mathrm{tv}(G, \alpha) = 1, \\ 0 & \text{otherwise.} \end{cases}$

An equivalent (and more succinct) way to code $\mathrm{tv}(F, \alpha)$:

1. $\mathrm{tv}(p, \alpha) = \alpha(p)$ for atoms p.
2. $\mathrm{tv}(\neg F, \alpha) = 1 - \mathrm{tv}(F, \alpha)$.
3. $\mathrm{tv}(F \wedge G, \alpha) = \min\{\mathrm{tv}(F, \alpha), \mathrm{tv}(G, \alpha)\}$.
4. $\mathrm{tv}(F \vee G, \alpha) = \max\{\mathrm{tv}(F, \alpha), \mathrm{tv}(G, \alpha)\}$.

Example. Compute $\mathrm{tv}((p \wedge (q \vee \neg p)), \alpha)$ on the assignment α that maps p to 0 and q to 1.

Solution. Firstly, we know the assignments are $\mathrm{tv}(p, \alpha) = 0$ and $\mathrm{tv}(q, \alpha) = 1$, which implies that $\mathrm{tv}(\neg p, \alpha) = 1$ and so

$$\mathrm{tv}((q \vee \neg p), \alpha) = \max\{\mathrm{tv}(q, \alpha), \mathrm{tv}(\neg p, \alpha)\} = \max\{1, 1\} = 1.$$

Therefore, we can deduce that

$$\mathrm{tv}((p \wedge (q \vee \neg p)), \alpha) = \min\{\mathrm{tv}(p, \alpha), \mathrm{tv}((q \vee \neg p), \alpha)\} = \min\{0, 1\} = 0.$$

Why do we need this precise definition of semantics?

1. It *specifies* how to evaluate formulas. If we disagree on the truth value of F under α, we just consult the definition.
2. This definition is *implemented* by the runtime environment of Python and Java to compute the values of Boolean expressions.
3. One can *prove* things about formulas or about code that processes formulas.

2.2 Terminology

We sometimes shorten $\mathrm{tv}(F, \alpha)$ and simply write $\alpha(F)$. E.g., $\mathrm{tv}(F \wedge G, \alpha) = 1$ may be written $\alpha(F \wedge G) = 1$. In case $\alpha(F) = 1$, we say any of the following:

- α makes F true;
- α satisfies F;
- α models F;
- also written $\alpha \models F$.
- The symbol $\models$ is called the *double-turnstile*.

2.3 Conditional

Many times it is convenient to use an extended syntax:

1. $\top$ and $\bot$ are formulas/
   - These symbols are called the *propositional constants*.
   - $\top$ is read *top* and $\bot$ is read *bottom*.
2. If F and G are formulas then so are $(F \to G)$ and $(F \leftrightarrow G)$.

- $\rightarrow$ is called the conditional (aka implementation).
- $\leftrightarrow$ is called the bi-conditional (aka bi-implication).

3. The semantics:

   - $\top$ is true under every assignment, and $\bot$ is false under every assignment.
   - $(F \leftrightarrow G)$ is true exactly when F and G have the same truth values.

The semantics/truth-table for $(F \rightarrow G)$ is the following.

F	G	$(F \rightarrow G)$
0	0	1
0	1	1
1	0	0
1	1	1

Notice that for $(F \rightarrow G)$ to be true, if F is true, then also G is true. If F is not true, then it does not matter whether G is true or not. Thus, we have noticed that $(F \rightarrow G)$ is equivalent to $(\neg F \vee G)$.

Note. Conditional is not the same as the programming construct "if condition is true then do instruction". Conditional does not mean that F causes G to be true.

Example. A robot has two sensor variables: `raining` and `carrying_umbrella` such that

```
ready = raining and carrying_umbrella
ready = not raining or carrying_umbrella
```

Assign a truth value to the Boolean variable `ready` that says whether or not the robot can go outside and not get wet.

Solution. The above code shows that `ready` only depends on `carrying_umbrella`. Hence, we have the following.

```
ready = carrying_umbrella
```

2.4 Connection with natural language

Although logic can be used to model propositions of natural language, there are important differences. In logic, semantics is determined by the assignment and the syntactic structure of the formula. In natural language, the semantics also depends on the context.

Be careful formalising English statements into logic.

1. "OR" in English:

- Sometimes it is inclusive ("you must be a member or rich to enter"), and so we would write $p \vee q$.
- Sometimes it is exclusive ("I will arrive now or later"), and so we would write $(p \vee q) \wedge \neg(p \wedge q)$.
- Sometimes it is unclear ("your error is in the program or the data").

2. "AND" in English:
   - "He threw the stone and the window broke".
   - "The window broke and he threw the stone".
   - Different meaning in common English, but the same in formal logic.

3 Validity and satisfiability

3.1 Valid formulas

Definition. A formula F is *valid* if every assignment satisfies F, i.e., if its truth-table always has value 1.

Valid formulas represent logical laws in the same way that $x + 0 = x$ is an arithmetic law.

Valid.

1. $p \vee \neg p$
2. $(p \wedge q) \leftrightarrow (q \wedge p)$
3. $(p \wedge (q \wedge r)) \leftrightarrow ((p \wedge q) \wedge r)$

Not valid.

1. $p \vee \neg q$
2. $(p \wedge (q \vee r)) \leftrightarrow ((p \wedge q) \vee r)$

3.2 Satisfiable formulas

Definition. A formula F is *satisfiable* if at least one assignment satisfies F, i.e., if its truth-table has at least one 1.

Examples.

1. $p \vee \neg q$ is satisfiable.
2. $p \wedge \neg p$ is not satisfiable.

3.3 Satisfiability problem

Definition. The *satisfiability problem* is to decide if a given propositional formula F is satisfiable (truth-table has at least one 1).

This can be solved, in principle, by checking all the rows of the truth-table for F.

Truth-table test for satisfiability:

Suppose F has n atoms. Then the truth-table has 2^n rows. So this algorithm runs in worst-case exponential time (in the size of F).

Suppose we can generate a table at the rate of one row per second. If $n = 80$, we will need 2^80 seconds to write out the table. This is about 2.8 million times the age of the universe. So we need a faster method.

4 Logical Consequences

An argument is a statement of the form "if these facts are true, then that fact must be true". Logic allows us to formalise what it means for an argument to be *logically* correct.

Example.

1. If it is raining then I carry an umbrella.
2. It is raining.
3. Conclude, I carry an umbrella.

It is a particular instance of the form of argument called *implication elimination* (aka **Modus Ponens**):

1. $p \to q$
2. p
3. Conclude, q

Example.

1. If it is hot, I wear a hat.
2. If it is windy, I wear a hat.
3. It is either hot or windy.
4. Conclude, I wear a hat.

It is a particular instance of the form of argument called *reasoning by cases* (aka **disjunction elimination**):

1. $h \to t$
2. $w \to t$
3. $h \vee w$
4. Conclude t

Example. The following argument is *not* logically correct.

1. If it is raining, then I take an umbrella.
2. I take an umbrella.
3. Conclude, it is raining.

Because, we can have "I take an umbrella" is true and "It is raining" is false, which would mean that statement 1 is true, statement 2 is true, but the conclusion statement 3 is false. This is called the *error of the converse*:

1. $p \to q$
2. q
3. Conclude, p

Definition. Say that F is a logical consequence of $E_1, \ldots, E_k$, if every assignment that makes all of the formulas E_i (for $1 \leq i \leq k$) true also makes F true.

We write this as

$$E_1, \ldots, E_k \models F.$$

In case $k = 0$ we write $\models F$, which means that every assignment makes F true, i.e., that F is *valid*.

Example. Show that $(p \to q), \neg q \models \neg p$.

p	q	$p \to q$	$\neg q$	$\neg p$
0	0	**1**	**1**	**1**
0	1	1	0	1
1	0	0	1	0
1	1	1	0	0

Example. Show that $(p \to q), \neg p \not\models \neg q$.

p	q	$p \to q$	$\neg p$	$\neg q$
0	0	1	1	1
0	1	**1**	**1**	**0**
1	0	0	0	1
1	1	1	0	0

5 Logical Equivalence

Formulas that "mean the same thing" are called *equivalent*. We now study common equivalences, also called *laws*.

Although the formulas $(A \wedge B)$ and $(B \wedge A)$ are syntactically different formulas, they mean the same thing. This is captured by the fact that they are *logically equivalent*.

Definition. Two formulas F and G are *logically equivalent* if,

$$\mathrm{tv}(F, \alpha) = \mathrm{tv}(G, \alpha)$$

for every assignment α. In words, F and G always (i.e., no matter the assignment) evaluate to the same truth value.

Table of Equivalences.

(Idempotent Laws)	$F \equiv (F \wedge F)$
	$F \equiv (F \vee F)$
(Commutative Laws)	$(F \wedge G) \equiv (G \wedge F)$
	$(F \vee G) \equiv (G \vee F)$
(Associative Laws)	$(F \wedge (G \wedge H)) \equiv ((F \wedge G) \wedge H)$
	$(F \vee (G \vee H)) \equiv ((F \vee G) \vee H)$
(Absorption Laws)	$(F \wedge (F \vee G)) \equiv F$
	$(F \vee (F \wedge G)) \equiv F$
(Distributive Laws)	$(F \wedge (G \vee H)) \equiv ((F \wedge G) \vee (F \wedge H))$
	$(F \vee (G \wedge H)) \equiv ((F \vee G) \wedge (F \vee H))$
(de Morgan's Laws)	$\neg(F \wedge G) \equiv (\neg F \vee \neg G)$
	$\neg(F \vee G) \equiv (\neg F \wedge \neg G)$
(Double Negation Law)	$\neg\neg F \equiv F$
(Validity Law)	$(F \vee \top) \equiv \top$
	$(F \wedge \top) \equiv F$
(Unsatisfiability Law)	$(F \vee \bot) \equiv F$
	$(F \wedge \bot) \equiv \bot$
(Constant Laws)	$\top \equiv (F \vee \neg F)$
	$\bot \equiv (F \wedge \neg F)$
(Negating constants Laws)	$\neg\top \equiv \bot$
	$\neg\bot \equiv \top$
(Conditional Law)	$(F \to G) \equiv (\neg F \vee G)$
(Bi-conditional Law)	$(F \leftrightarrow G) \equiv ((F \to G) \wedge (G \to F))$

We should verify, using truth-tables or reasoning, that each of these pairs of formulas is an equivalence.

Example. Absorption Law $F \wedge (F \vee G) \equiv F$.

F	G	$F \wedge (F \vee G)$	F
0	0	0	0
0	1	0	0
1	0	1	1
1	1	1	1

We can verify new equivalences by deducing them from known equivalences.

Example. Prove $(p \to q) \equiv (\neg q \to \neg p)$.

$$\begin{aligned}
(p \to q) &\equiv (\neg p \vee q) && \text{Conditional Law} \\
&\equiv (q \vee \neg p) && \text{Commutative Law for } \vee \\
&\equiv (\neg\neg q \vee \neg p) && \text{Double Negation Law} \\
&\equiv (\neg q \to \neg p) && \text{Conditional Law}
\end{aligned}$$

Fact. If F is a subformula of H, and $F \equiv G$, then H is equivalent to the formula that results by substituting any occurrence of F in H by G. This is called the *Substitution Rule*.

6 Normal forms and modeling problems in logic

In this section we use the original syntax of propositional logic, i.e., the only operators are $\neg$, $\wedge$ and $\vee$. If we have a formula with any other operator, we should first get rid of it and replacing by equivalent formulas. E.g., replace $(p \to q)$ by $(\neg p \vee q)$; replace $\perp$ by $(p \wedge \neg p)$.

6.1 Normal forms: NNF

Definition. A formula F is in *negation normal form (NNF)* if negations only occur immediately in front of atoms.

For example, $(\neg q \vee p)$ is in NNF, $\neg(q \wedge \neg p)$ is not.

Theorem. For every formula F there is an equivalent formula in NNF.

Algorithm ("push negation inwards"): substitute in F every occurrence of a subformula of the form.

$$\begin{array}{rl} \neg\neg G \text{ by } G & \text{Double Negation Law} \\ \neg(G \wedge H) \text{ by } (\neg G \vee \neg H) & \text{de Morgan's Law} \\ \neg(G \vee H) \text{ by } (\neg G \wedge \neg H) & \text{de Morgan's Law} \end{array}$$

until no such subformulas occer, and return the result.

Example. Put $\neg(\neg p \wedge (\neg(r \wedge s) \vee q))$ into NNF.

$$\begin{aligned} \neg(\neg p \wedge (\neg(r \wedge s) \vee q)) &\equiv \neg(\neg p \wedge ((\neg r \vee \neg s) \vee q)) && \text{de Morgan's Law} \\ &\equiv \neg\neg p \vee \neg((\neg r \vee \neg s) \vee q) && \text{de Morgan's Law} \\ &\equiv p \vee (\neg(\neg r \vee \neg s) \wedge \neg q) && \text{Double Negation Law and de Morgan's Law} \\ &\equiv p \vee ((\neg\neg r \wedge \neg\neg s) \wedge \neg q) && \text{de Morgan's Law} \\ &\equiv p \vee ((r \wedge s) \wedge \neg q) && \text{Double Negation Law} \end{aligned}$$

6.2 Normal forms: CNF

Definition.

- A *literal* is an atomic formula or the negation of an atomic formula.
  i.e., a literal has the form p or $\neg p$; note that $\neg\neg p$ is not a literal.
- A *clause* is a literal or a disjunction of literals.
  e.g., both $(p \vee \neg q \vee r)$ and $\neg r$ are clauses. But $(p \vee q) \wedge r$ is not a clause.

- A formula F is in *conjunctive normal form (CNF)* if it is a clause or a conjunction of clauses.

Example. The following are in CNF:

$$(p \vee \neg q \vee r) \wedge (\neg p \vee r) \wedge q \wedge \neg p$$
$$p \wedge q$$
$$p \vee q$$

However, $(p \wedge q) \vee r$ is not.

Say we have two variables x, y.

Write a formula that says that "the variables have the same value".

$$(x \wedge y) \vee (\neg x \wedge \neg y).$$

Now write this in CNF:

- Its negation says that x, y have different values, which we can easily write:

$$(x \wedge \neg y) \vee (\neg x \wedge y)$$

- So let's negate to get back to the meaning we want:

$$\begin{aligned}
\neg((x \wedge \neg y) \vee (\neg x \wedge y)) &\equiv \neg(x \wedge \neg y) \wedge \neg(\neg x \wedge y) \\
&\equiv (\neg x \vee \neg\neg y) \wedge (\neg\neg x \vee \neg y) \\
&\equiv (\neg x \vee y) \wedge (x \vee \neg y) \\
&\equiv (x \to y) \wedge (y \to x) \\
&\equiv (x \leftrightarrow y)
\end{aligned}$$

  which is in CNF.

Another way to see why the CNF formula

$$(\neg x \vee y) \wedge (x \vee \neg y)$$

says "the variables x and y take the same values"...

Each clause eliminates an assignment which does not satisfy the specification.

1. $(\neg x \vee y)$ eliminates the assignment where x is true and y is false.
2. $(x \vee \neg y)$ eliminates the assignment where x is false and y is true.

Theorem. For every formula F there is an equivalent formula in CNF.

Proof. Here is the algorithm:

1. Put F in NNF, call it F'.
2. Substitute in F' each occurrence of a subformula of the form

$$((H \wedge I) \vee G) \text{ or } (G \vee (H \wedge I))$$
$$\text{by } ((G \vee H) \wedge (G \vee I))$$

   until no such subformulas occur, and return the result.

7 The problem of finding cliques

Definition (Clique). A *clique* in a graph is a set of vertices all of which are adjacent to each other. That is for a graph $G = (V, E)$, $E \subseteq V \times V$.

The CLIQUE problem

Input: undirected graph $G = (V, E)$ and an integer $K \geq 1$.
Output: "Yes" if the graph has a clique of size K, "No" otherwise.

- A *clique* (aka *complete graph*) of an undirected graph (V, E) is a set $C \subseteq V$ of vertices such that every two (distinct) vertices in C are adjacent.
- The size of C is the number of vertices in it, written $|C|$.

A naive algorithm would work as follows: for every set S of K vertices, check if there are edges between all the vertices in S.

There are $\binom{|V|}{K}$ such sets, and so this takes time approximately $|V|^K$, which is exponential in the size of the input.

Solving CLIQUE with logic:

1. Given input: $G = (V, E)$, $K \in \mathbb{Z}^+$
2. Construct a formula $\Phi_{V,E,K}$
3. Check if the formula $\Phi_{V,E,K}$ is satisfiable.
4. If it is satisfiable, return "Yes, there is a clique of size K in the graph", otherwise return "No".

7.1 Solving CLIQUE with logic

Given $G = (V, E)$ and $K \in \mathbb{Z}^+$.

Idea: Introduce a variable for each vertex, say x_i for $1 \leq i \leq |V|$, and write a formula expressing "the true variables form a clique in (V, E)" of size K."

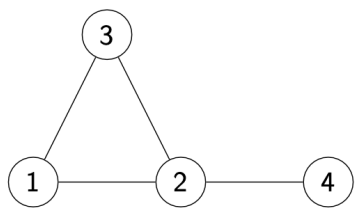
i.e., express in logic that

1. There are exactly K many true variables.
2. Every pair of true variables correspond to an edge in graph.

Then: the satisfying assignments of the formula identify the cliques of size K in (V, E).

So, checking if the formula is satisfiable will check if there is a clique of size K in (V, E).

7.2 Running Example



Introduce four variables x_1, x_2, x_3, x_4.

How do we express "There are exactly $K = 3$ many true variables" in logic?

The idea is to express that there is a subset S of $\{1, 2, 3, 4\}$ of size 3 such that the variables in S are true and the variables out of S are false.

S can be $\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 3, 4\}$, $\{2, 3, 4\}$, so the formula is:

$$(x_1 \wedge x_2 \wedge x_3 \wedge x_4) \vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4) \vee (x_1 \wedge \neg x_2 \wedge x_3 \wedge x_4) \vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge x_4)$$

In general, to express "there are exactly K many true variables":

For a set $S \subseteq \{1, 2, \ldots, |V|\}$ of indices:

- $\bigwedge_{i \in S} x_i$ says that all the variables indexed by S are true.
- $\bigwedge_{i \notin S} \neg x_i$ says that all the variables not indexed by S are false.

- So the following says what we want:

$$\bigvee_{|S|=K} \left(\bigwedge_{i \in S} x_i \right) \wedge \left(\bigwedge_{i \notin S} \neg x_i \right)$$

Express "Every pair of true variables correspond to an edge in the graph":

$$\bigwedge_{(i,j) \notin E} \neg(x_i \wedge x_j)$$

Putting this together, given input $G = (V, E)$ and size K, build the formula $\Phi_{V,E,K}$:

$$\left(\bigvee_{|S|=K} \left(\bigwedge_{i \in S} x_i \right) \wedge \left(\bigwedge_{i \notin S} \neg x_i \right) \right) \wedge \left(\bigwedge_{(i,j) \notin E} \neg(x_i \wedge x_j) \right)$$

In this specific example of $S = \{1, 2, 3, 4\}$ and $K = 3$, we have

$$\big((x_1 \wedge x)2 \wedge x_3 \wedge \neg x_4) \vee (x_1 \wedge x_2 \wedge \neg x_3 \wedge x_4) \vee (x_1 \wedge \neg x_3 \wedge x_3 \wedge x_4) \vee (\neg x_1 \wedge x_2 \wedge x_3 \wedge x_4)\big)$$
$$\wedge$$
$$\big(\neg(x_1 \wedge x_4) \wedge \neg(x_3 \wedge x_4)\big)$$

The size of this formula is $O(|V|^K + |V|^2)$, which is exponential in the size of the input.